Guia de leitura dos logs e métricas

Todos os serviços escrevem em stdout → `docker compose logs <serviço>`. Formato comum: `<timestamp UTC> <serviço> <NÍVEL> chave=valor ...` (o Prometheus usa o formato próprio dele, `ts=… level=… msg=…`).

1. Nginx — uma linha por requisição

```
2026-11-02T23:14:05+00:00 nginx INFO msec=1793661245.856 request_id=0148397b806478487e2ef85a9c175
  method=GET uri=/ status=200 upstream=172.20.0.2:8080, 172.20.0.3:8080 node=rust-b
  upstream_status=504, 200 req_time=1.001 upstream_time=1.001, 0.000
```

Campo	Significado	Nesta linha
`msec`	horário unix com ms (o ISO só tem segundos)	—
`request_id`	id gerado pelo Nginx, enviado ao nó em `X-Request-Id` e devolvido ao cliente	procurar o mesmo id no log do Rust
`status`	o que o **cliente** recebeu	200: o usuário não viu erro
`upstream`	endereços **tentados**, em ordem. Dois = houve retry	tentou A (`.2`) e depois B (`.3`)
`node`	nome do nó que respondeu por último (cabeçalho `X-Node`)	rust-b
`upstream_status`	status de cada tentativa	`504` = timeout de conexão em A; `200` em B
`req_time`	tempo total visto pelo cliente	1,001 s = o `proxy_connect_timeout` de 1 s
`upstream_time`	tempo de cada tentativa	1,001 s em A, 0 em B

Padrões para reconhecer:

Padrão	Situação
`upstream_status=200` , um endereço	normal
`upstream_status=503, 200`	nó **não pronto** recusou; refeito no outro (E5)
`upstream_status=504, 200` , `req_time≈1.0`	nó **parado/inacessível**: estourou o connect timeout; refeito (E1)
`upstream_status=504, 200` , `req_time≈3.0`	nó **lento** além do read timeout (E2b)
`status=500` , um endereço	erro da aplicação; **não** refeito, de propósito (E4)
`status=502` , `upstream=rust_pool` ou vazio	**nenhum** nó disponível (os dois fora)

Linhas do **error log** (mesmo stream):

Mensagem	Significado
`upstream timed out (110: Operation timed out) while connecting to upstream`	connect timeout (1 s) — nó parado: o IP não responde

Mensagem	Significado
`upstream server temporarily disabled`	atingiu `max_fails=3`; nó fora do pool por `fail_timeout=10s`
`rust-a could not be resolved (3: Host not found)`	a re-resolução (`resolve`) não achou o nome: contêiner parado
`client: 172.20.0.1` / `client: 172.20.0.x`	`.1` = gateway (requisição do host); outro IP = um contêiner (ex.: watchdog, teste sintético)

2. Rust (rust-a / rust-b)

```
2026-11-02T23:00:48.185Z rust-b INFO request_id=f4cae6d9f804864bab4fcfbbc8ab4edb method=GET path=
2026-11-02T23:01:15.374Z rust-a INFO evento=startup porta=8080 ready=1 atraso_ms=0
2026-11-02T23:14:04.700Z rust-a WARN evento=shutdown motivo=sinal_recebido
2026-11-02T23:00:19.300Z rust-a WARN request_id=- admin=ready valor=0
```

- Nível: `INFO` (2xx/3xx), `WARN` (4xx e ações admin), `ERROR` (5xx).
- Sondas (`/health/*`) e `/metrics` só são registradas **se falharem** — senão o log seria dominado pelas sondas a cada 5 s e o tráfego de usuário sumiria.
- `request_id=-`: a chamada não passou pelo Nginx (ex.: `scripts/admin.sh`, sonda direta).
- `evento=startup` marca cada início de processo → conta reinícios.
- `latency_ms` mede só o tempo **dentro do nó** (inclui o atraso artificial).

3. Watchdog

```
... watchdog INFO evento=startup intervalo_s=5.0 timeout_s=2.0 limite_falhas=3 alvos=['rust-a', '
... watchdog INFO ciclo resumo rust-a=UP rust-b=UP sintetico-nginx=UP
... watchdog WARN alvo=rust-a tipo=readiness resultado=falha status=503 erro=http_503 falhas_cons
... watchdog ERROR alvo=rust-a tipo=readiness transicao=DOWN motivo=3_falhas_consecutivas ultimo_
... watchdog WARN alvo=rust-a tipo=readiness transicao=UP apos_falhas=4 latencia_ms=2
```

`erro=`	Significado
`http_503`	o nó respondeu e disse "não estou pronto" (processo vivo)
`prazo_total_2s_excedido(dns/conexao/leitura)`	não respondeu em 2 s: parado, lento, DNS lento ou rede
`URLError:[Errno_111]_Connection_refused`	host existe, porta fechada (processo morto, contêiner de pé)

- `ciclo resumo` a cada 5 s é a **prova de vida** do watchdog; se parar de aparecer, o monitor parou. Filtrar: `docker compose logs watchdog | grep -v resumo`.
- `tipo=sintetico` = teste do caminho do usuário via Nginx; `tipo=readiness` = sonda direta ao nó.
- `apos_falhas=N` na volta a UP diz quantas sondagens falharam no total.

4. Correlação de uma requisição (passo a passo)

```
rid=$(curl -s -D - -o /dev/null localhost:8080/ | tr -d '\r' | awk -F': ' 'tolower($1)=="x-reques
docker compose logs --no-log-prefix nginx rust-a rust-b | grep "$rid"
```

Saída real (`evidencias/preliminar/04_balanceamento_20req.txt`): uma linha do Rust e uma do Nginx com o mesmo id. Correlacione **pelo id**, não pelo horário: o relógio do host salta alguns segundos (`docs/achados.md` #7).

5. Métricas: onde olhar no Prometheus (http://127.0.0.1:9090)

Pergunta	PromQL
Quem está UP para o watchdog?	`watchdog_target_up`
Distribuição do tráfego por nó	`sum by (node)(rate(app_requests_total{route="/"}[1m]))`
Taxa de erro de usuário	`sum(rate(app_requests_total{route=~"/|/erro",status=~"5..",status!="503"}[2m])) / sum(rate(app_requests_total{route=~"/|/erro"}[2m]))`
p95 por nó	`histogram_quantile(0.95, sum by (node,le) (rate(app_request_duration_seconds_bucket[2m])))`
Idade da última sondagem	`time() - max(watchdog_last_probe_timestamp_seconds)`
O Prometheus alcança cada alvo?	`up`
Houve reinício?	`changes(app_start_time_seconds[10m])`
Latência das sondas	`watchdog_probe_duration_seconds`

Atalho no terminal: `python3 scripts/promq.py query '<PromQL>'` e `python3 scripts/promq.py alerts`.